

TodoTec v2

Sección 9 — Flujogramas de Procesos

Detalle completo de los archivos, programas y base de datos que intervienen en cada proceso del sistema.

Proceso	Archivo frontend	Router backend	Tablas afectadas
Registro de Compra	src/pages/Compras.jsx	routers/purchases.py	purchases, purchase_items, products, product_variants, acco
Registro de Venta	src/pages/Ventas.jsx	routers/sales.py	sales, sale_items, product_variants, company_config, accou
Aporte de Capital	src/pages/Contabilidad.jsx	routers/accounting.py	capital_contributions, accounting_entries

Flujo 1 — Registro de Compra

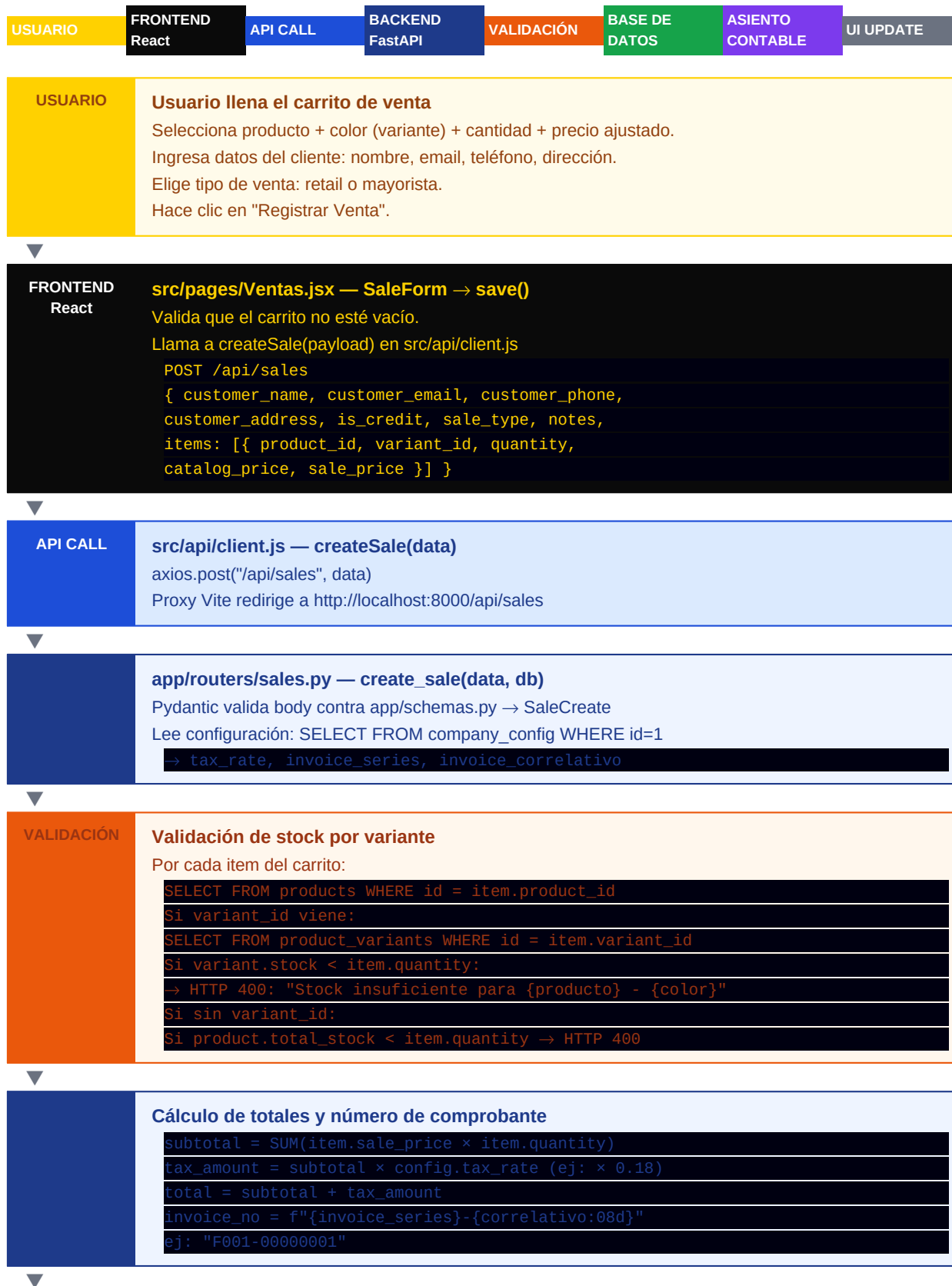
El usuario registra una compra con múltiples productos y variantes de color. El sistema valida el efectivo disponible, crea los registros en base de datos y genera el asiento contable.





Flujo 2 — Registro de Venta

El usuario registra una venta desde el carrito del admin. El sistema valida el stock por variante de color, genera el comprobante, reduce el stock y crea dos asientos contables (venta + costo de ventas).

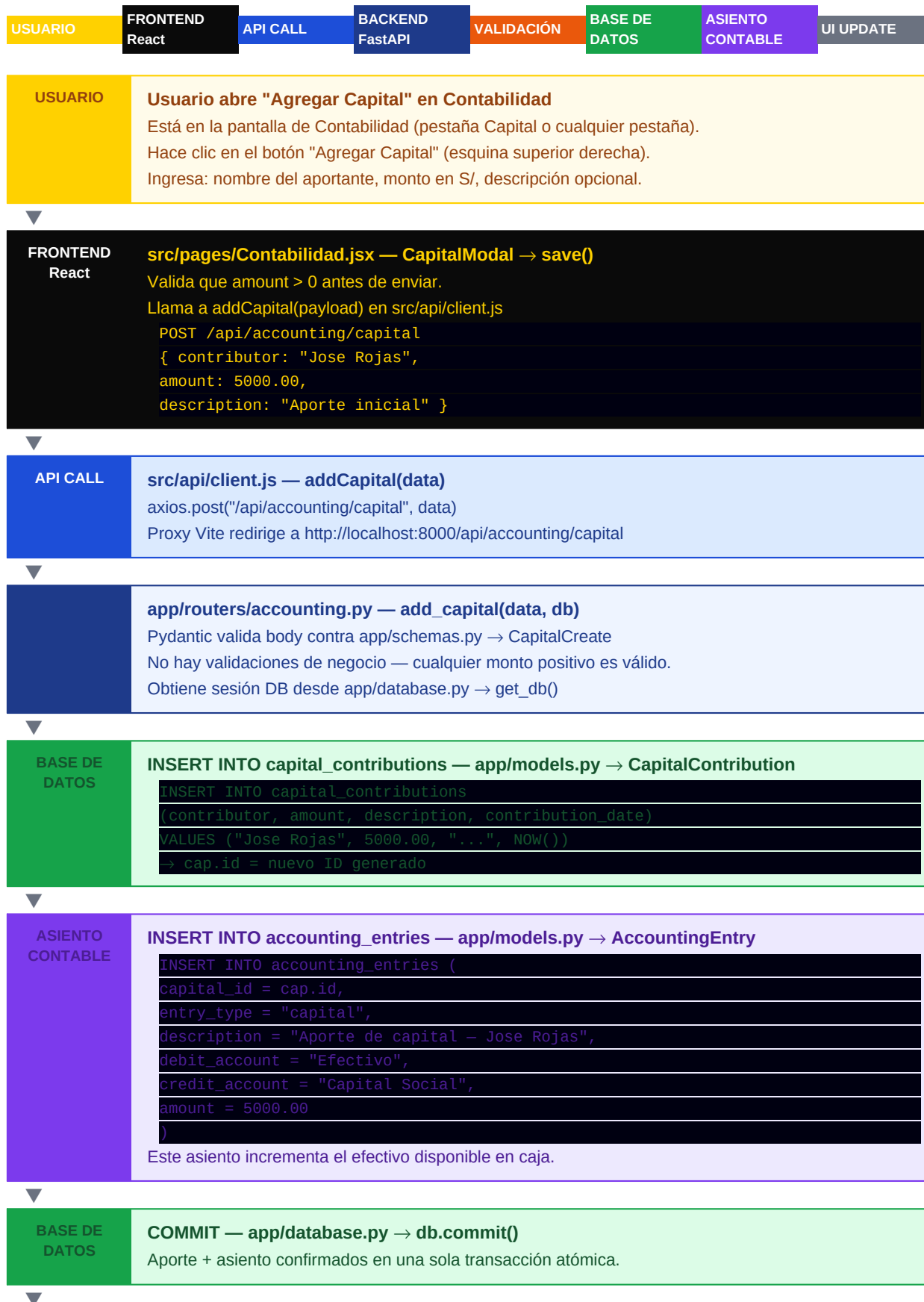


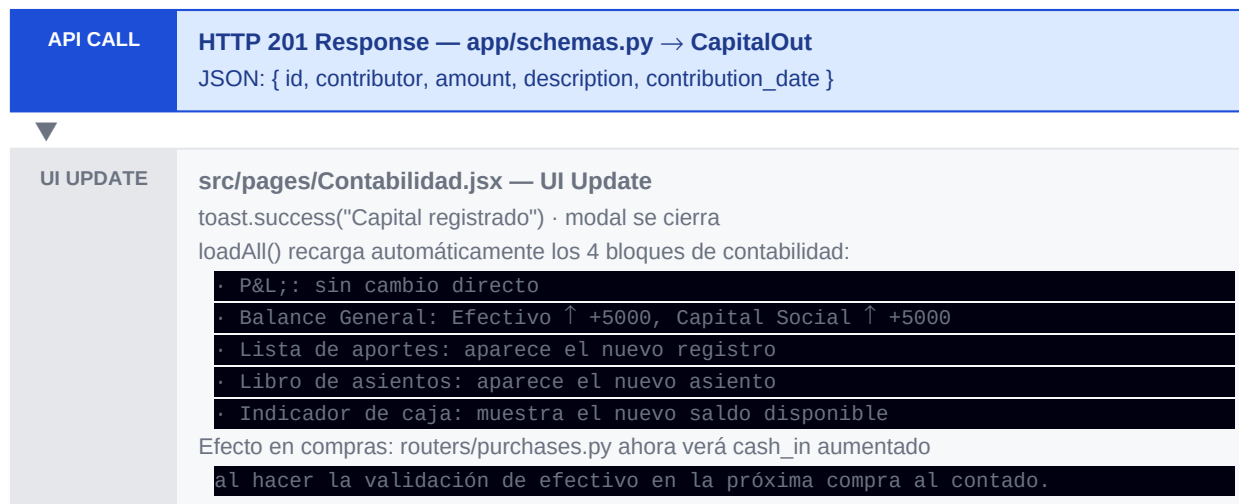
BASE DE DATOS	INSERT INTO sales — app/models.py → Sale <pre> INSERT INTO sales (sale_date, customer_name, customer_email, customer_phone, customer_address, subtotal, tax_amount, total, is_credit, status, invoice_number, sale_type, notes) → sale.id = nuevo ID </pre>
BASE DE DATOS	INSERT INTO sale_items — app/models.py → SaleItem Por cada item del carrito: <pre> unit_cost = product.unit_cost ← costo prom. ponderado de purchase_items INSERT INTO sale_items (sale_id, product_id, variant_id, quantity, catalog_price, sale_price, unit_cost, subtotal) </pre>
BASE DE DATOS	UPDATE product_variants stock — app/models.py → ProductVariant Con variant_id: <pre> UPDATE product_variants SET stock = stock - quantity WHERE id = variant_id </pre> Sin variant_id: <pre> Busca ira variante con stock suficiente y reduce </pre>
BASE DE DATOS	UPDATE company_config — app/models.py → CompanyConfig <pre> UPDATE company_config SET invoice_correlativo = invoice_correlativo + 1 WHERE id = 1 </pre>
ASIENTO CONTABLE	Asiento 1: Venta — accounting_entries <pre> INSERT INTO accounting_entries (sale_id, entry_type = "venta", description = "Venta F001-XXXXXXX", Contado → debit="Efectivo" credit="Ventas" Crédito → debit="Cuentas por Cobrar" credit="Ventas" amount = total (con IGV)) </pre>
ASIENTO CONTABLE	Asiento 2: COGS — accounting_entries <pre> cogs = SUM(product.unit_cost × item.quantity) INSERT INTO accounting_entries (sale_id, entry_type = "cogs", description = "Costo de ventas F001-XXXXXXX", debit = "Costo de Ventas", credit = "Inventarios", amount = cogs) </pre>
BASE DE DATOS	COMMIT — app/database.py → db.commit() Transacción atómica: venta + stock – + 2 asientos en un solo commit.

API CALL	HTTP 201 Response — app/schemas.py → SaleOut JSON con la venta completa: invoice_number, items, totales, cliente
UI UPDATE	src/pages/Ventas.jsx — UI Update toast.success("Venta registrada") · modal se cierra load() — recarga la lista de ventas Usuario puede abrir el comprobante (■■■) → InvoiceModal → imprimir/PDF store/app/catalog/ refleja stock reducido al recargar la tienda

Flujo 3 — Aporte de Capital

El usuario registra un aporte de capital de uno de los socios. El sistema crea el registro del aporte y genera automáticamente el asiento contable que incrementa el efectivo disponible, habilitando compras al contado.





Todos los flujos usan transacciones atómicas: si algún paso falla, ningún dato se guarda. Los asientos contables se generan en el mismo commit que los datos operacionales.